

Applied Artificial Intelligence | Hochschule Hof

Prof. Dr. Lars Ackermann-Igl

REINFORCEMENT LEARNING

Complete Exam-Ready Study Guide

- ◆ All Theory with Deep Explanations
 - ◆ Bellman Equation Mastered
- ◆ Q-Learning Step-by-Step Calculations
- ◆ Mock Exam Questions with Model Answers
 - ◆ Most Likely Exam Questions Covered

01 | What is
RL?

02 | Key
Concepts

03 |
Q-Learning

04 | Bellman
Equation

05 | Deep
Q-Learning

06 | Mock
Exam Q&A;

07 | Extra
Questions

SECTION 1 — WHAT IS REINFORCEMENT LEARNING?

Core Definition

Reinforcement Learning (RL) is a machine learning paradigm where an **agent** learns to make decisions by interacting with an **environment**. Unlike supervised learning (which requires labeled data), the agent learns through **trial and error** — receiving **rewards** for good actions and **penalties** for bad ones. The agent's goal is to find a strategy (called a **policy**) that **maximises the cumulative reward** over time.

Why RL? — When Supervised Learning Fails

Supervised learning needs labeled examples of correct answers. In many real-world scenarios this is impossible or impractical:

Outcome-centric judgement	We know the final result (win/lose) but cannot label every individual action as correct (e.g., in chess, was move 23 correct?).
Delayed rewards	The consequence of an action may only appear much later (e.g., a wrong chess move may only cost the game 10 moves later).
Long-term consequences	Each action changes the environment's future state, creating chains of cause and effect (e.g., robot arm movements).

Applications of RL

- Game playing (Atari, AlphaGo, Chess) • Autonomous driving & robotics
- Supply chain & logistics optimisation • Training Large Language Models (LLMs/RLHF)

■ **EXAM TIP:** RL is the only paradigm covered in this course that doesn't require labeled data at all. The exam often asks you to contrast it with supervised and unsupervised learning.

SECTION 2 — KEY CONCEPTS IN REINFORCEMENT LEARNING

Agent	The learner and decision-maker. It perceives the environment and chooses actions. Examples: a chess program, a robot, a self-driving car.
Environment	Everything the agent interacts with. It defines the rules and responds to the agent's actions by providing a new state and a reward. Examples: a chess board, the physical world, a video game.
State (s)	A snapshot of the environment at a given time. It contains all information the agent needs to make a decision (e.g., piece positions on a chess board, sensor readings of a robot).
Action (a)	A choice made by the agent that affects the environment. The set of all possible actions is called the action space A . Examples: move left, move right, jump, pass ball.
Reward (R)	A scalar feedback signal from the environment. Positive reward = good action, negative reward (penalty) = bad action. The agent aims to MAXIMISE cumulative reward, NOT just immediate reward.
Policy (π)	The strategy the agent uses to decide actions. Given a state s , the policy $\pi(s)$ returns the action to take. A perfect policy always picks the optimal action in every state.
Trajectory	A sequence of states and actions: $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots)$. The agent experiences many trajectories during learning. Cumulative reward = sum of all rewards along a trajectory.

The RL Feedback Loop — How the Agent Learns

- Step 1.** Agent observes current state s from the environment.
- Step 2.** Agent selects action a based on its current policy.
- Step 3.** Agent performs action a in the environment.
- Step 4.** Environment transitions to new state s' and emits reward R .
- Step 5.** Agent updates its policy based on (s, a, R, s') .
- Step 6.** Repeat from Step 1 until the task is complete (episode ends).

Key insight: RL is based on *trial and error*. The agent does NOT know the correct action in advance — it discovers good behaviour purely by experiencing consequences of its actions.

SECTION 3 — Q-LEARNING (VALUE-BASED RL)

Value-Based vs. Policy-Based Methods

Value-Based (e.g., Q-Learning)	Policy-Based (e.g., REINFORCE)
Learns a value function $Q(s,a)$ that estimates how good each action is in each state.	Learns a policy function $\pi(a s)$ that gives probabilities over actions directly.
Selects the action with the highest Q-value (highest expected reward).	Samples actions from the learned probability distribution .
Output: Q-values (numbers)	Output: Action probabilities

Q-Values and the Q-Table

Q-Learning uses a **Q-Table** to store Q-values. A Q-value $Q(s, a)$ represents the **maximum expected cumulative reward** the agent can achieve when it takes action **a** in state **s** and then follows the optimal policy thereafter.

Example Q-Table (Snake Game):

State	Left	Right	Up	Down
Heading Left	0	0	0	1
Apple Right	0	5	0	0
Apple Left	2	0	0	0
Heading Up	-1	0	1	0

Highlighted cells = best action (highest Q-value) for that state.

The Q-Learning Algorithm — Step by Step

Step 1	Initialise Q-Table Create a table with all states as rows and all actions as columns. Fill every cell with 0 (or a small random number).
Step 2	Choose action a for state s Select the action with the highest Q-value for the current state. Initially all values are 0, so choose randomly to explore.
Step 3	Perform action a Execute the chosen action in the environment. The environment moves to a new state s' .
Step 4	Observe reward R Receive the reward signal from the environment (e.g., +10 for eating an apple, -10 for dying).
Step 5	Update Q-Table via Bellman Equation Use the Bellman equation to update $Q(s, a)$ with the new information (see Section 4).
Step 6	Repeat Set $s = s'$ and go back to Step 2. Repeat until the episode ends, then start a new episode.

Exploration vs. Exploitation (Epsilon-Greedy)

Problem: If the agent always picks the highest Q-value action, it may never explore better alternatives and get stuck in a suboptimal strategy.

Solution — Epsilon (ϵ) greedy:

- With probability ϵ : choose a **random action** (EXPLORE)
- With probability $1-\epsilon$: choose the **best known action** (EXPLOIT)

Schedule: Start with high epsilon (lots of exploration at the beginning) and gradually reduce it as training progresses (more exploitation once the agent has learned something useful).

SECTION 4 — THE BELLMAN EQUATION (MOST IMPORTANT!)

The Complete Formula

$$Q_{\text{new}}(s, a) = Q(s, a) + \alpha \cdot [R(s, a) + \gamma \cdot \max_{a'} Q'(s', a') - Q(s, a)]$$

Every Component Explained

$Q_{\text{new}}(s, a)$

The UPDATED Q-value for taking action a in state s . This is what we write back into the Q-table after each step.

$Q(s, a)$

The CURRENT Q-value (before update) — how good we currently think action a is in state s . This is the old value stored in the Q-table.

alpha (Learning Rate)

Controls how fast the agent learns. $\alpha=0$ means no learning (Q never changes). $\alpha=1$ means replace the old estimate entirely with the new one. Typically $0 < \alpha < 1$.

$R(s, a)$ — Immediate Reward

The actual reward received from the environment after taking action a in state s . This is the instant feedback — did we just earn +10 (ate apple) or -10 (hit wall)?

gamma (Discount Factor)

How much the agent cares about FUTURE rewards vs. immediate rewards. $\gamma=0$: only care about right now. $\gamma=1$: care equally about all future rewards. Typically set to 0.9 or 0.99 in practice.

$\max_{a'} Q'(s', a')$ — Future Potential

The best Q-value available in the NEXT state s' . This estimates the maximum future reward the agent can obtain from the next state onwards. 'max' means we look at ALL possible actions a' in s' and take the highest value.

$[R + \gamma \cdot \max_{a'} Q'(s', a') - Q(s, a)]$ — Temporal Difference Error

Also called the TD-error. This is the DIFFERENCE between what we expected (old Q-value) and what we now estimate is possible (immediate reward + discounted future value). If positive: our old estimate was too pessimistic → increase Q. If negative: too optimistic → decrease Q. If zero: Q was already correct.

■ **EXAM TIP:** The Bellman equation is THE most tested topic. You must be able to: (1) Write the full formula from memory. (2) Name and explain every symbol. (3) Calculate a Q-value update given numbers.

SECTION 5 — DEEP Q-LEARNING (DQN)

The Problem with Standard Q-Learning: State Explosion

State explosion issue: In real-world problems (e.g., autonomous driving), the number of possible states is astronomically large. A pixel-based game could have millions of distinct screen configurations — each requiring its own row in the Q-table. This quickly exceeds any computer's memory.

Example: A simple snake game on a small 4×4 grid → a few hundred states (manageable). An Atari game at full resolution → trillions of possible states (impossible to store).

The Deep Q-Learning Solution

Instead of storing a Q-table, Deep Q-Learning uses a **neural network** to *approximate* the Q-value function.

Standard Q-Learning	Deep Q-Learning (DQN)
Input: (state s , action a) pair	Input: state s (only)
Lookup $Q(s,a)$ in table	Neural network computes $Q(s,a)$ for all actions simultaneously
Output: single Q-value	Output: Q-values for ALL actions at once
Update: write new value into table cell	Update: backpropagation (adjust NN weights)
Fails with large/continuous state spaces	Handles large, continuous, complex state spaces (e.g., images)

Why is DQN still a VALUE-based approach?

Because the neural network outputs Q-values (one per action) — not a probability distribution. The agent still selects actions by choosing the one with the highest Q-value. A policy-based approach would output probabilities directly.

SECTION 6 — MOCK EXAM: TASK 6 WITH FULL MODEL ANSWERS

Task 6 — Reinforcement Learning | Total: 15 Points | Subtasks: 6a (3P) + 6b (4P) + 6c (8P)

EXAM QUESTION 6a [3 Points]

Explain the concept of reinforcement learning by explaining the relations between agents, the environment and rewards.

✓ MODEL ANSWER

Reinforcement Learning (RL) is a machine learning approach in which an **agent** learns to make decisions by interacting with an **environment** through trial and error, guided by **reward** signals.

The Agent is the learner and decision-maker. It perceives the current state of the environment and selects an action according to its current policy. The agent has no prior knowledge of which actions are good — it must discover this through experience.

The Environment defines the world in which the agent operates. It sets the rules, boundaries, and dynamics. After the agent takes an action, the environment transitions to a new state and sends back a reward signal.

Reward is the feedback mechanism linking the agent and environment. A positive reward signals a good action (e.g., +10 for eating an apple in Snake); a negative reward (penalty) signals a bad action (e.g., -10 for hitting a wall). The agent's goal is to maximise its **cumulative reward** over time — not just the immediate reward at each step.

The cycle: Agent observes state $s \rightarrow$ selects action $a \rightarrow$ environment transitions to new state s' and emits reward $R \rightarrow$ agent learns from $(s, a, R, s') \rightarrow$ repeat.

Key distinction from supervised learning: No labeled 'correct action' is ever provided. The agent discovers what is correct purely from the reward signal.

EXAM QUESTION 6b [4 Points]

Explain each underlined component of the Bellman equation:

$$Q_{\text{new}}(s, a) = Q(s, a) + \underline{\alpha} \cdot [\underline{R(s, a)} + \underline{\gamma} \cdot \underline{\max_{a'} Q'(s', a')} - Q(s, a)]$$

✓ MODEL ANSWER

alpha (Learning Rate):

Controls how quickly the agent updates its Q-values. $\alpha = 0$ means the agent learns nothing new (ignores new information). $\alpha = 1$ means the Q-value is completely replaced by the new estimate. Typically set between 0 and 1 (e.g., $\alpha = 0.1$).

R(s, a) — Immediate Reward:

The reward actually received from the environment after taking action a in state s . This is the instant feedback signal — the direct consequence of the last action taken.

gamma (Discount Factor):

Determines how much future rewards are valued relative to immediate rewards. $\gamma = 0$: the agent is completely myopic (only cares about the next reward). $\gamma = 1$: the agent weighs all future rewards equally. A value like $\gamma = 0.9$ means a reward 2 steps away is worth 0.81 times what it would be immediately.

max $Q'(s', a')$ — Future Potential (Maximum Future Q-value):

The best Q-value achievable from the next state s' . By looking at all possible actions a' in the next state s' , we take the maximum. This represents our best estimate of how much total future reward the agent can collect from the next state onwards if it acts optimally.

EXAM QUESTION 6c [8 Points]

Q-Learning with a deterministic environment. States: $S = \{s1, s2\}$, Actions: $A = \{a1, a2\}$, Learning rate: $\alpha = 1$, Discount factor: $\gamma = 0.5$.

Experienced sequence:

Step 1: $s1 \xrightarrow{a1, r=2} s2$

Step 2: $s2 \xrightarrow{a1, r=4} s1$

Q-Table is initialised to all zeros. Describe the algorithm and fill in the Q-table after each step.

✓ MODEL ANSWER**Algorithm Description:**

Q-Learning is a value-based reinforcement learning algorithm that iteratively updates a Q-Table by applying the Bellman equation. Starting from an all-zero Q-Table, the agent takes actions, observes rewards and next states, and updates the Q-value for the (state, action) pair it just experienced. After enough iterations, the Q-Table converges to the true optimal Q-values.

INITIAL Q-TABLE (all zeros):

State/Actions	a1	a2
s1	0	0
s2	0	0

STEP 1 CALCULATION — $s1 \rightarrow a1 \rightarrow s2$ (reward=2)

We are updating $Q(s1, a1)$ because we took action $a1$ in state $s1$.

Identify all values:

- Current state: $s = s1$
- Action taken: $a = a1$
- Reward received: $R(s1, a1) = 2$
- Next state: $s' = s2$
- $\max Q'(s2, a') = \max(Q(s2, a1), Q(s2, a2)) = \max(0, 0) = 0$ [Q-Table is all zeros]
- Learning rate: $\alpha = 1$
- Discount factor: $\gamma = 0.5$

Apply Bellman equation:

$$Q_{\text{new}}(s1, a1) = Q(s1, a1) + \alpha \cdot [R(s1, a1) + \gamma \cdot \max Q'(s2, a') - Q(s1, a1)]$$

$$Q_{\text{new}}(s1, a1) = 0 + 1 \cdot [2 + 0.5 \cdot 0 - 0]$$

$$Q_{\text{new}}(s1, a1) = 0 + 1 \cdot [2 + 0 - 0]$$

$$Q_{\text{new}}(s1, a1) = 2$$

Q-TABLE AFTER STEP 1:

State/Actions	a1	a2
s1	2	0
s2	0	0

STEP 2 CALCULATION — s2 → a1 → s1 (reward=4)

Now we are in state s2, taking action a1, receiving reward 4, landing in s1.

We are updating $Q(s2, a1)$.

Identify all values:

- Current state: $s = s2$
- Action taken: $a = a1$
- Reward received: $R(s2, a1) = 4$
- Next state: $s' = s1$
- $\max Q'(s1, a') = \max(Q(s1, a1), Q(s1, a2)) = \max(2, 0) = 2$ [from Step 1 update]
- Learning rate: $\alpha = 1$
- Discount factor: $\gamma = 0.5$

Apply Bellman equation:

$$Q_{\text{new}}(s2, a1) = Q(s2, a1) + \alpha \cdot [R(s2, a1) + \gamma \cdot \max Q'(s1, a') - Q(s2, a1)]$$

$$Q_{\text{new}}(s2, a1) = 0 + 1 \cdot [4 + 0.5 \cdot 2 - 0]$$

$$Q_{\text{new}}(s2, a1) = 0 + 1 \cdot [4 + 1 - 0]$$

$$Q_{\text{new}}(s2, a1) = 5$$

Q-TABLE AFTER STEP 2 (FINAL):

State/Actions	a1	a2
s1	2	0
s2	5	0

■ **EXAM TIP:** When $\alpha=1$, the formula simplifies to: $Q_{\text{new}}(s,a) = R(s,a) + \gamma \cdot \max Q'(s',a')$. The old Q-value is completely discarded! Always check alpha first.

SECTION 7 — MOST LIKELY ADDITIONAL EXAM QUESTIONS

EXAM QUESTION EQ1 [~3 Points]

What is the difference between reinforcement learning and supervised learning? Give one example scenario for each where one approach is clearly more suitable.

✓ MODEL ANSWER

Supervised Learning trains on labeled data — each training example has a correct answer provided by a human. The model learns to map inputs to these known outputs.

Example: Image classification — thousands of images labeled 'cat' or 'dog' allow the model to learn the mapping directly.

Reinforcement Learning has no labeled correct answers. The agent discovers what is good through trial and error, guided only by reward signals.

Example: Playing chess — we cannot label each of the millions of possible board positions with the 'correct move'. Instead, the agent learns by playing games and receiving +1 for winning, -1 for losing.

Key difference: Supervised learning has immediate, explicit feedback for every training example. RL has delayed, sparse feedback (rewards only come after actions, often much later).

EXAM QUESTION EQ2 [~3 Points]

Explain exploration vs. exploitation in Q-Learning. What is epsilon and how is it typically used?

✓ MODEL ANSWER

Exploration means the agent tries new, potentially unknown actions to discover whether they yield better rewards. Without exploration, the agent could miss optimal strategies.

Exploitation means the agent uses its current best knowledge (highest Q-values) to select actions. Without exploitation, the agent never benefits from what it has learned.

The epsilon (ε) greedy strategy balances both: with probability epsilon, take a random action (explore); with probability 1-epsilon, take the action with the highest Q-value (exploit).

Epsilon scheduling: Start with epsilon close to 1 (almost all exploration at the beginning when the agent knows nothing), then gradually reduce it over training (more exploitation once the agent has learned useful Q-values). At the end, epsilon approaches 0 (nearly pure exploitation).

EXAM QUESTION EQ3 [~3 Points]

What is the difference between Q-Learning and Deep Q-Learning? What problem does Deep Q-Learning solve?

✓ MODEL ANSWER

Standard Q-Learning stores all Q-values in an explicit lookup table (Q-Table). Each row represents a state, each column an action. This works well for small, discrete state spaces.

The problem — state explosion: In real-world tasks, the number of possible states can be astronomically large (e.g., billions of distinct pixel combinations in a video game). Storing a Q-Table becomes impossible.

Deep Q-Learning (DQN) replaces the Q-Table with a neural network that approximates the Q-value function. The network takes a state as input and outputs Q-values for all actions simultaneously. Rather than looking up a table entry, the network generalises — it recognises patterns and applies learned Q-values to similar but previously unseen states. Weights are updated via backpropagation.

EXAM QUESTION EQ4 [~2 Points]

What is the role of the discount factor gamma in the Bellman equation? What happens when gamma = 0 and when gamma = 1?

✓ MODEL ANSWER

The discount factor gamma controls how much the agent values future rewards compared to immediate rewards.

gamma = 0: The agent is completely short-sighted — it only cares about the immediate reward and ignores all future consequences. This is rarely useful in practice because the agent never plans ahead.

gamma = 1: The agent values all future rewards equally, regardless of how far in the future they occur. This requires the task to be finite (otherwise the sum of rewards could be infinite).

In practice (e.g., gamma = 0.9): A reward 2 steps away is worth $0.9^2 = 0.81$ of its face value. This encourages the agent to find paths to reward quickly rather than accumulating small rewards slowly.

EXAM QUESTION EQ5 [~4 Points]

A Q-Learning agent is in state A and takes action 'right', receiving reward $R=5$ and arriving in state B. The Q-Table currently contains: $Q(A, \text{right})=2$, $Q(B, \text{up})=10$, $Q(B, \text{down})=6$. Use $\alpha=0.5$ and $\gamma=0.9$. Calculate $Q_{\text{new}}(A, \text{right})$.

✓ MODEL ANSWER

Step 1: Identify values:

- $Q(s, a) = Q(A, \text{right}) = 2$
- $R(s, a) = 5$
- $\max Q'(B, a') = \max(Q(B, \text{up}), Q(B, \text{down})) = \max(10, 6) = 10$
- $\alpha = 0.5$, $\gamma = 0.9$

Step 2: Apply Bellman equation:

$$Q_{\text{new}}(A, \text{right}) = 2 + 0.5 \cdot [5 + 0.9 \cdot 10 - 2]$$

$$Q_{\text{new}}(A, \text{right}) = 2 + 0.5 \cdot [5 + 9 - 2]$$

$$Q_{\text{new}}(A, \text{right}) = 2 + 0.5 \cdot 12$$

$$Q_{\text{new}}(A, \text{right}) = 2 + 6 = 8$$

The Q-value increased from 2 to 8 because the new information (reward 5 + discounted future value of 9) suggests state A → right is more valuable than we previously thought.

EXAM QUESTION EQ6 [~2 Points]

Why is Deep Q-Learning still considered a VALUE-BASED approach rather than a policy-based approach?

✓ MODEL ANSWER

Even though DQN uses a neural network (like policy-based approaches), it is still value-based because:

- The neural network outputs **Q-values** (one per action) — numerical estimates of expected future reward.
- The agent then selects the action with the **highest Q-value** (greedy selection).
- A true policy-based approach would output a **probability distribution** over actions (e.g., 70% chance left, 20% chance right, 10% chance up). The action is then sampled from this distribution.

The distinction: value-based = output is reward-centric numbers; policy-based = output is probability-centric numbers.

QUICK REFERENCE CHEAT SHEET — REINFORCEMENT LEARNING

$$Q_{\text{new}}(s, a) = Q(s, a) + \alpha \cdot [R(s, a) + \gamma \cdot \max_{a'} Q'(s', a') - Q(s, a)]$$

Symbol	Name	Meaning	Range
$Q_{\text{new}}(s,a)$	Updated Q-value	New value written to Q-table	any real
$Q(s,a)$	Current Q-value	Old Q-table entry (before update)	any real
α	Learning rate	How fast to update (0=none, 1=full)	0 to 1
$R(s,a)$	Immediate reward	Reward from environment right now	defined by task
γ	Discount factor	Weight on future vs immediate reward	0 to 1
$\max Q'(s',a')$	Future potential	Best Q-value in next state	any real
ϵ	Exploration rate	Probability of random action (explore)	0 to 1

Q-Learning Algorithm — At a Glance

- 1. Initialise Q-Table with all zeros (states x actions)
- 2. Choose action a in current state s (highest Q-value or random via epsilon-greedy)
- 3. Perform action $a \rightarrow$ receive reward R and new state s'
- 4. Apply Bellman equation to compute $Q_{\text{new}}(s, a)$
- 5. Update Q-Table: $Q(s, a) \leftarrow Q_{\text{new}}(s, a)$
- 6. Set $s = s'$ and repeat from step 2

RL uses	trial and error + reward signals (NO labeled data)
Q-Table stores	$Q(s,a)$ = expected cumulative reward for action a in state s
Agent selects	action with HIGHEST Q-value (greedy / epsilon-greedy)
DQN solves	state explosion by replacing Q-table with neural network
DQN is still	value-based (outputs Q-values, not probabilities)
$\alpha=1$ simplifies to	$Q_{\text{new}} = R + \gamma \cdot \max_{a'} Q'(s',a')$

EXAM STRATEGY: (1) Always write the full Bellman equation first — then identify each symbol from the given parameters. (2) When $\alpha=1$, the old Q-value cancels: $Q_{\text{new}} = R + \gamma \cdot \max Q'$. (3) For $\max Q'(s', a')$, look at ALL entries in the row of the next state s' and pick the largest. (4) If all Q-values in next state are 0, the equation becomes $Q_{\text{new}} = Q(s,a) + \alpha \cdot (R - Q(s,a))$. (5) Explain RL by always mentioning: agent, environment, state, action, reward, policy.